

# COMP6237 Data Mining

## Lecture 5: Recommendation Systems

Zhiwu Huang

[Zhiwu.Huang@soton.ac.uk](mailto:Zhiwu.Huang@soton.ac.uk)

Lecturer (Assistant Professor) @ VLC of ECS  
University of Southampton

Lecture slides available here:

<http://comp6237.ecs.soton.ac.uk/zh.html>

[Book time with Zhiwu Huang: Office Hour](#)

(Thanks to Prof. Jonathon Hare and Dr. Jo Grundy for providing the lecture materials used to develop the slides.)

# Recommendation Systems – Problem



|                | Life is Beautiful | Seven Samurai | Joker | Schindler's List | The Pianist | City Of God |
|----------------|-------------------|---------------|-------|------------------|-------------|-------------|
| <b>Lee</b>     | 2.5               | 3.5           | 3.0   | 3.5              | 3.0         | 2.5         |
| <b>Sofia</b>   | 3.0               | 3.5           | 1.5   | 5.0              | 3.0         | 3.5         |
| <b>Miley</b>   | 2.5               | 3.0           | NaN   | 3.5              | 4.0         | NaN         |
| <b>Justina</b> | NaN               | 3.5           | 3.0   | 4.0              | 4.5         | 3.0         |
| <b>Donald</b>  | 3.0               | 4.0           | 2.0   | 3.0              | 3.0         | 2.0         |
| <b>Mickey</b>  | 3.0               | 4.0           | NaN   | 5.0              | 3.0         | 3.5         |
| <b>Tristan</b> | NaN               | 4.5           | NaN   | 4.0              | NaN         | 1.0         |

Context: This table is a mini “Netflix”. We have 7 users, 6 films, and some of their 1–5 star ratings. Many entries are missing – we don’t yet know how every user feels about every film.

# Recommendation Systems – Problem

Problem: The user-item rating table is **incomplete**. Using only the observed ratings, can we **predict the missing entries** — in particular **Miley's ratings** for the two unseen films “**Joker**” and “**City Of God**”? In other words: what should we **recommend to Miley next**?

|         | Life is Beautiful | Seven Samurai | Joker | Schindler's List | The Pianist | City Of God |
|---------|-------------------|---------------|-------|------------------|-------------|-------------|
| Lee     | 2.5               | 3.5           | 3.0   | 3.5              | 3.0         | 2.5         |
| Sofia   | 3.0               | 3.5           | 1.5   | 5.0              | 3.0         | 3.5         |
| Miley   | 2.5               | 3.0           | NaN   | 3.5              | 4.0         | NaN         |
| Justina | NaN               | 3.5           | 3.0   | 4.0              | 4.5         | 3.0         |
| Donald  | 3.0               | 4.0           | 2.0   | 3.0              | 3.0         | 2.0         |
| Mickey  | 3.0               | 4.0           | NaN   | 5.0              | 3.0         | 3.5         |
| Tristan | NaN               | 4.5           | NaN   | 4.0              | NaN         | 1.0         |

Today we'll focus on **Miley**: predict her missing ratings for **Joker** and **City of God**, then recommend the ones with high predicted ratings.

# Recommendation Systems– Content-based Approach



|         | Life is Beautiful | Seven Samurai | Joker | Schindler's List | The Pianist | City Of God |
|---------|-------------------|---------------|-------|------------------|-------------|-------------|
| Miley   | 2.5               | 3.0           | NaN   | 3.5              | 4.0         | NaN         |
| romance | 0.9               | 0.2           | 0.1   | 0.4              | 0.5         | 0.2         |
| action  | 0.2               | 0.9           | 0.8   | 0.4              | 0.3         | 0.9         |

**Content-based approach** recommend items using item features. Here each film has two features: **romance** and **action** (between 0 and 1). The system learns Miley's **taste profile** from her rated films, then recommends new films whose (romance, action) features are closest to that profile.



# Recommendation Systems– Content-based Approach

|       |                | Life is Beautiful | Seven Samurai | Joker | Schindler's List | The Pianist | City Of God |
|-------|----------------|-------------------|---------------|-------|------------------|-------------|-------------|
| $y$   | <b>Miley</b>   | 2.5               | 3.0           | NaN   | 3.5              | 4.0         | NaN         |
| $x_1$ | <b>romance</b> | 0.9               | 0.2           | 0.1   | 0.4              | 0.5         | 0.2         |
| $x_2$ | <b>action</b>  | 0.2               | 0.9           | 0.8   | 0.4              | 0.3         | 0.9         |

Each film can be described by the vector  $X = [1 \quad x_1 \quad x_2]$  (1 is for the bias term)

Learn 2D parameter vector  $\theta$ , where  $\theta^T X$  gives the number of stars for each user.

$\theta = [0 \quad 1 \quad 0]$  for someone who really likes romance films

# Recommendation Systems– Content-based Approach

|       |                | Life is Beautiful | Seven Samurai | Joker | Schindler's List | The Pianist | City Of God |
|-------|----------------|-------------------|---------------|-------|------------------|-------------|-------------|
| $y$   | <b>Miley</b>   | 2.5               | 3.0           | NaN   | 3.5              | 4.0         | NaN         |
| $x_1$ | <b>romance</b> | 0.9               | 0.2           | 0.1   | 0.4              | 0.5         | 0.2         |
| $x_2$ | <b>action</b>  | 0.2               | 0.9           | 0.8   | 0.4              | 0.3         | 0.9         |

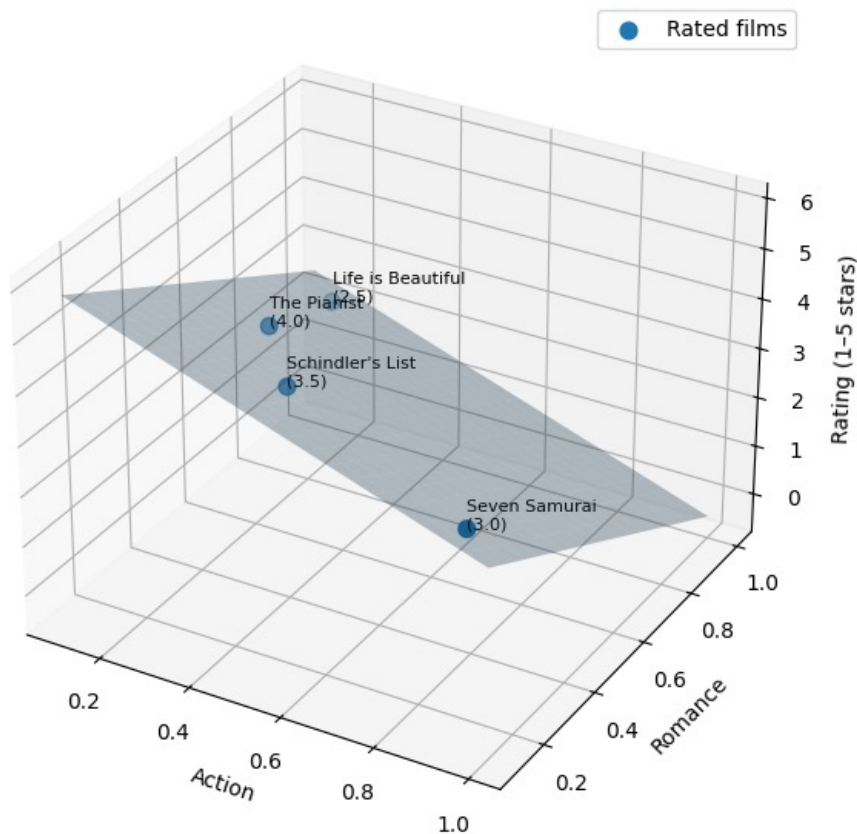
Use **Linear Regression** to find user parameter vector  $\theta$  where  $m$  is the number of films rated by that user

$$\min_{\theta} \frac{1}{m} \sum_{i=1}^m (\theta^T X_i - y_i)^2 \quad (1)$$

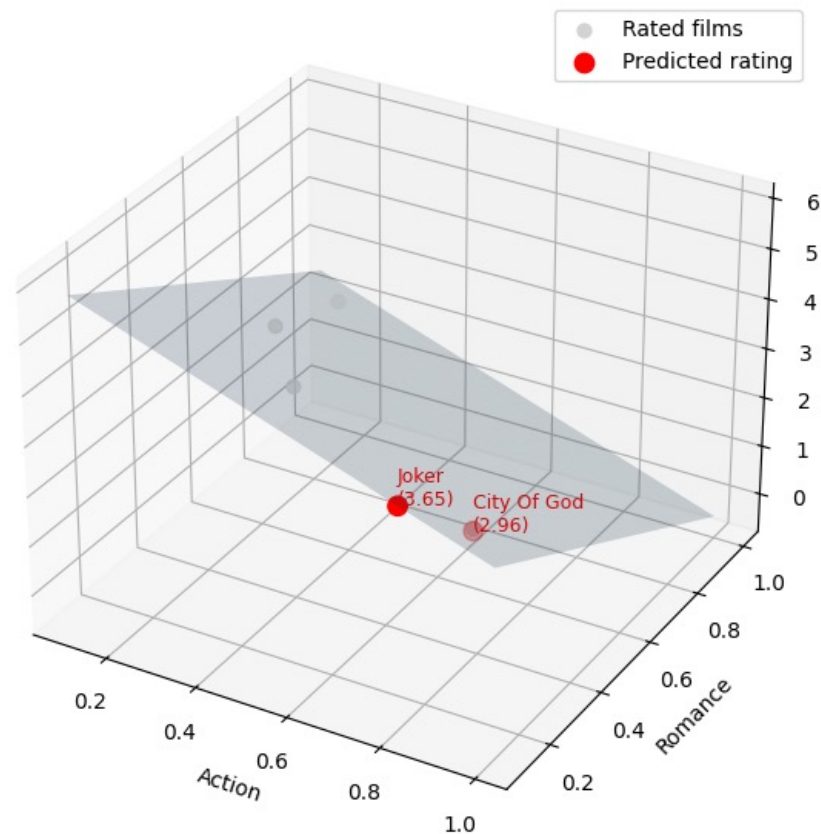
Can also use Bayesian classifiers, MLPs, etc.

# Recommendation Systems– Content-based Approach

Miley's ratings and fitted regression plane



Miley: predicted ratings for Joker & City Of God



Demo: Content-based recommendation

# Recommendation Systems – Collaborative Approach

|         | Life is Beautiful | Seven Samurai | Joker | Schindler's List | The Pianist | City Of God |
|---------|-------------------|---------------|-------|------------------|-------------|-------------|
| Lee     | 2.5               | 3.5           | 3.0   | 3.5              | 3.0         | 2.5         |
| Sofia   | 3.0               | 3.5           | 1.5   | 5.0              | 3.0         | 3.5         |
| Miley   | 2.5               | 3.0           | NaN   | 3.5              | 4.0         | NaN         |
| Justina | NaN               | 3.5           | 3.0   | 4.0              | 4.5         | 3.0         |
| Donald  | 3.0               | 4.0           | 2.0   | 3.0              | 3.0         | 2.0         |
| Mickey  | 3.0               | 4.0           | NaN   | 5.0              | 3.0         | 3.5         |
| Tristan | NaN               | 4.5           | NaN   | 4.0              | NaN         | 1.0         |

**Collaborative (user-based) systems use ratings from similar users.** We find users whose rating patterns look like **Miley's** (e.g. Lee or Sofia). If they liked a film Miley hasn't rated yet, such as "**Joker**" or "**City Of God**", we predict Miley will like it too and recommend it.



# Recommendation Systems – Collaborative Filtering

Collaborative filtering uses a range of approaches to accomplish this task

- ▶ Neighbourhood based approach
- ▶ Model based approach
- ▶ Hybrid (Neighbourhood and model) based approach

This lecture will cover the Neighborhood based approach

# Recommendation Systems – Collaborative Filtering

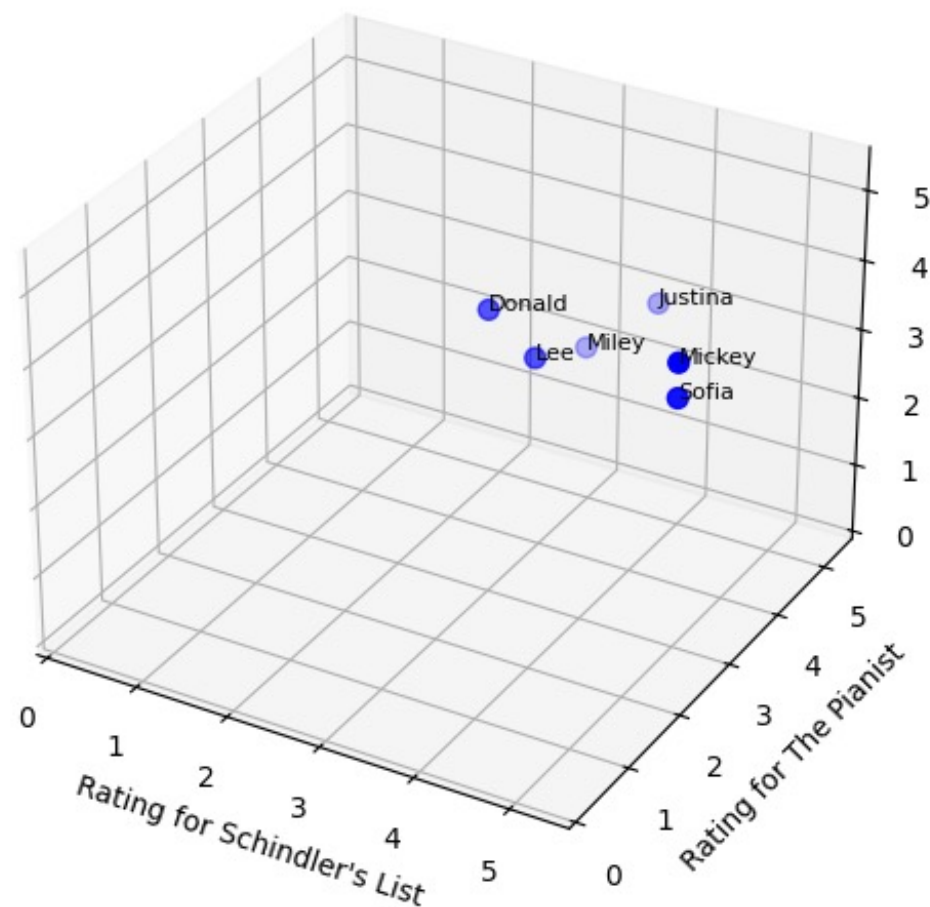
|         | Life is Beautiful | Seven Samurai | Joker | Schindler's List | The Pianist | City Of God |
|---------|-------------------|---------------|-------|------------------|-------------|-------------|
| Lee     | 2.5               | 3.5           | 3.0   | 3.5              | 3.0         | 2.5         |
| Sofia   | 3.0               | 3.5           | 1.5   | 5.0              | 3.0         | 3.5         |
| Miley   | 2.5               | 3.0           | NaN   | 3.5              | 4.0         | NaN         |
| Justina | NaN               | 3.5           | 3.0   | 4.0              | 4.5         | 3.0         |
| Donald  | 3.0               | 4.0           | 2.0   | 3.0              | 3.0         | 2.0         |
| Mickey  | 3.0               | 4.0           | NaN   | 5.0              | 3.0         | 3.5         |
| Tristan | NaN               | 4.5           | NaN   | 4.0              | NaN         | 1.0         |

**Question:** When can we say two users are “neighbours”?  
 What do we need to define before that statement even makes sense?

# Collaborative Filtering – Feature

|         | Seven Samurai | Schindler's List | The Pianist |
|---------|---------------|------------------|-------------|
| Lee     | 3.5           | 3.5              | 3.0         |
| Sofia   | 3.5           | 5.0              | 3.0         |
| Miley   | 3.0           | 3.5              | 4.0         |
| Justina | 3.5           | 4.0              | 4.5         |
| Donald  | 4.0           | 3.0              | 3.0         |
| Mickey  | 4.0           | 5.0              | 3.0         |
| Tristan | 4.5           | 4.0              | NaN         |

Users in 3D movie-rating space

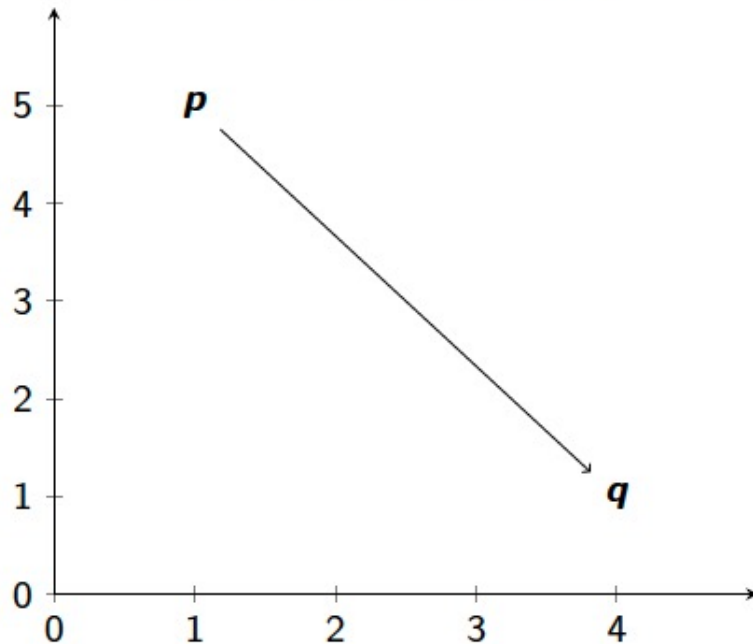


**Question:** where is Tristan in this 3D space?

# Collaborative Filtering – Distance

There are a number of ways to measure distance in feature space:

► Euclidean Distance:



$\mathbf{p}$  and  $\mathbf{q}$  are N-dimensional feature vectors,

$$\mathbf{p} = [p_1, p_2, \dots, p_N],$$

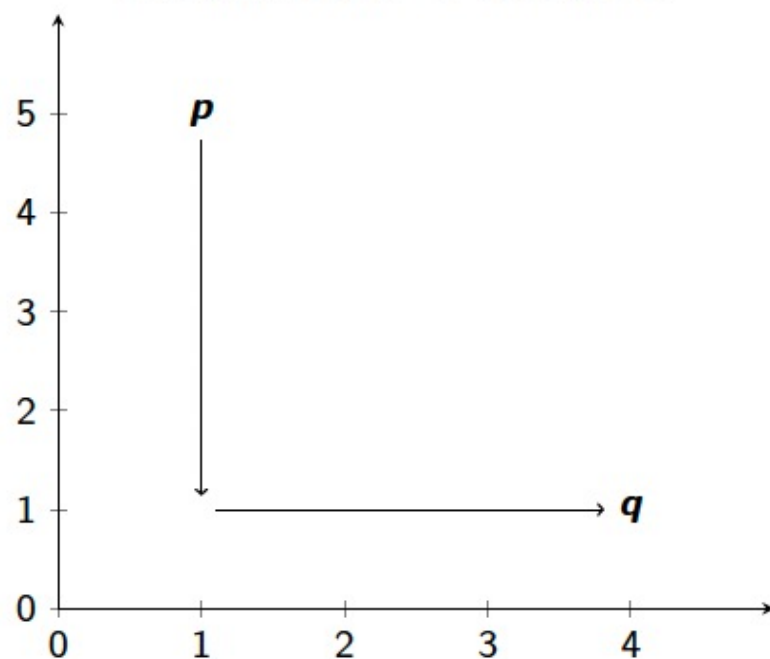
$$\mathbf{q} = [q_1, q_2, \dots, q_N]$$

Euclidean distance:

$$\|\mathbf{p} - \mathbf{q}\| = \sqrt{\sum_{i=1}^N (q_i - p_i)^2}$$

# Collaborative Filtering – Distance

## ► Manhattan Distance:



$\mathbf{p}$  and  $\mathbf{q}$  are N-dimensional feature vectors,

$$\mathbf{p} = [p_1, p_2, \dots, p_N],$$

$$\mathbf{q} = [q_1, q_2, \dots, q_N]$$

Manhattan distance:

$$\|\mathbf{p} - \mathbf{q}\|_1 = \sum_{i=1}^N |q_i - p_i|$$



# Collaborative Filtering – User Similarity

There are many ways to compute similarity based on Euclidean distance

We *could* chose:

$$sim_{L2}(x, y) = \frac{1}{1 + \sqrt{\sum_{i \in I_{xy}} (r_{x,i} - r_{y,i})^2}}$$

where  $r_{x,i}$  is the rating from user  $x$  for item  $i$

$I_{xy}$  is set of items rated by both  $x$  and  $y$

*i.e.* when the distance is 0, the similarity is 1, but when the distance is large, similarity  $\rightarrow 0$

# Collaborative Filtering – User Similarity

Can also use cosine similarity

$$sim_{cos}(x, y) = \frac{\sum_{i \in I_{xy}} r_{x,i} r_{y,i}}{\sqrt{\sum_{i \in I_{xy}} r_{xi}^2} \sqrt{\sum_{i \in I_{xy}} r_{yi}^2}}$$

Only performed over the items which are rated by both users

# Collaborative Filtering – User Similarity

Alternatively, calculate correlation of users, based on ratings they share

Using Pearson's Correlation: standard measure of dependence between two related variables.

$$\text{sim}_{\text{Pearson}}(x, y) = \frac{\sum_{i \in I_{xy}} (r_{x,i} - \bar{r}_x)(r_{y,i} - \bar{r}_y)}{\sqrt{\sum_{i \in I_{xy}} (r_{x,i} - \bar{r}_x)^2 \sum_{i \in I_{xy}} (r_{y,i} - \bar{r}_y)^2}}$$

Where  $\bar{r}_x$  is average rating user  $x$  gave for all items in  $I_{xy}$

# Collaborative Filtering – User Filtering

We now have a set of measures for computing the similarity between users

Produce a ranked list of best matches to a target user. Typically want the top- $N$  users

May only want to consider a subset of users, i.e. those who rated a particular item.

# Collaborative Filtering – User-based Recommending

Now we have a list of similar users, how can we recommend items?

Predict rating  $r_{u,i}$  of item  $i$  by user  $u$  as an aggregation of the ratings of item  $i$  by users similar to  $u$

$$r_{u,i} = \text{aggr}_{\hat{u} \in U}(r_{\hat{u},i})$$

Where  $U$  is the set of *top* users most similar to  $u$  that rated item  $i$   
Multiply the score by the similarity of the user

Normalise by sum of similarities (otherwise items rated more often will dominate)

$$r_{u,i} = \frac{\sum_{\hat{u} \in U} \text{sim}(u, \hat{u}) r_{\hat{u},i}}{\sum_{\hat{u} \in U} |\text{sim}(u, \hat{u})|}$$

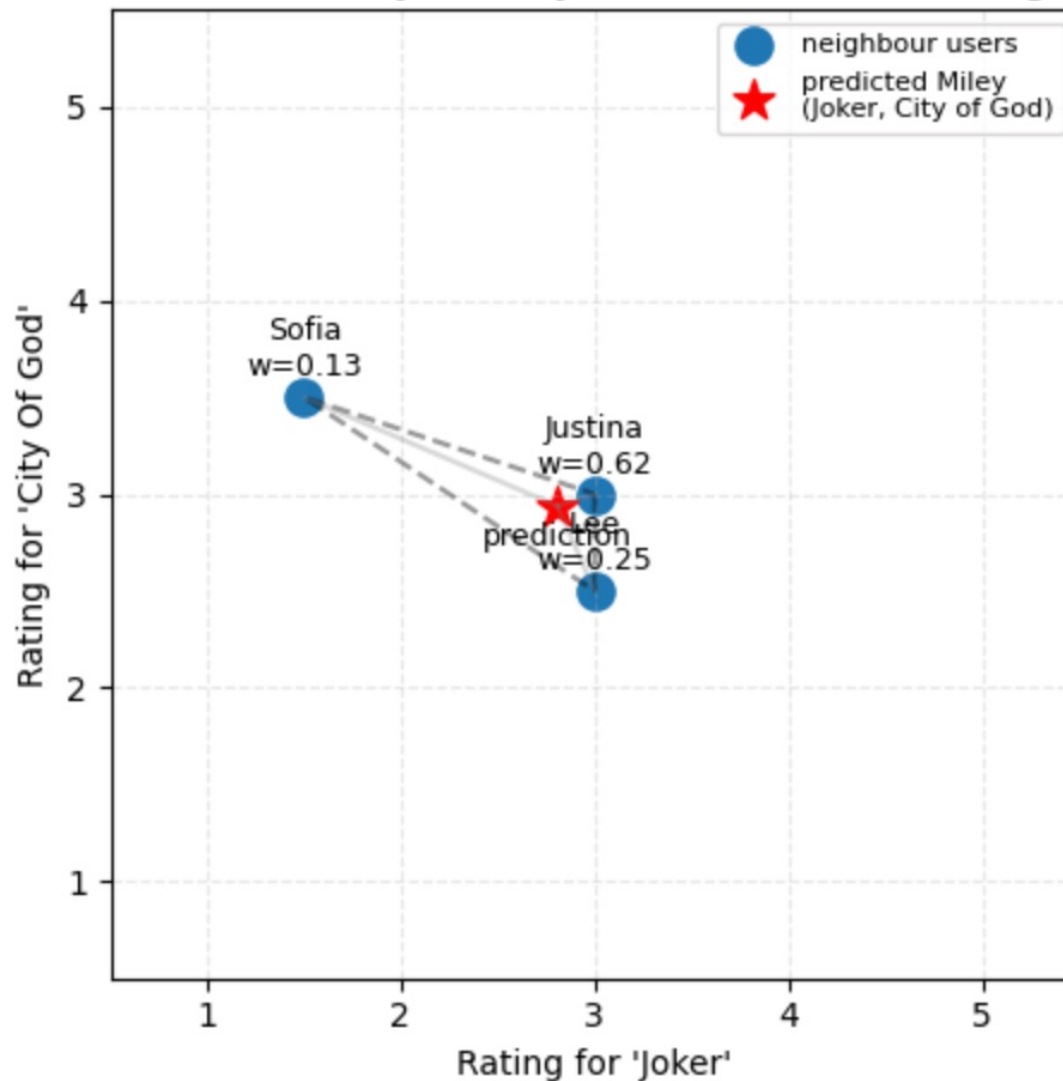
This is *User Based Filtering*

Target user: Miley

Neighbours (used for barycentric combination):

|         |            |            |               |
|---------|------------|------------|---------------|
| Justina | sim=1.000, | Joker=3.0, | CityOfGod=3.0 |
| Lee     | sim=0.405, | Joker=3.0, | CityOfGod=2.5 |
| Sofia   | sim=0.205, | Joker=1.5, | CityOfGod=3.5 |

User-based CF: Miley as barycentric combo of 3 neighbours



Barycentric prediction for Miley:

Joker: 2.81  
City Of God: 2.94

Demo: User-based recommendation



# Collaborative Filtering – Item-based Approach

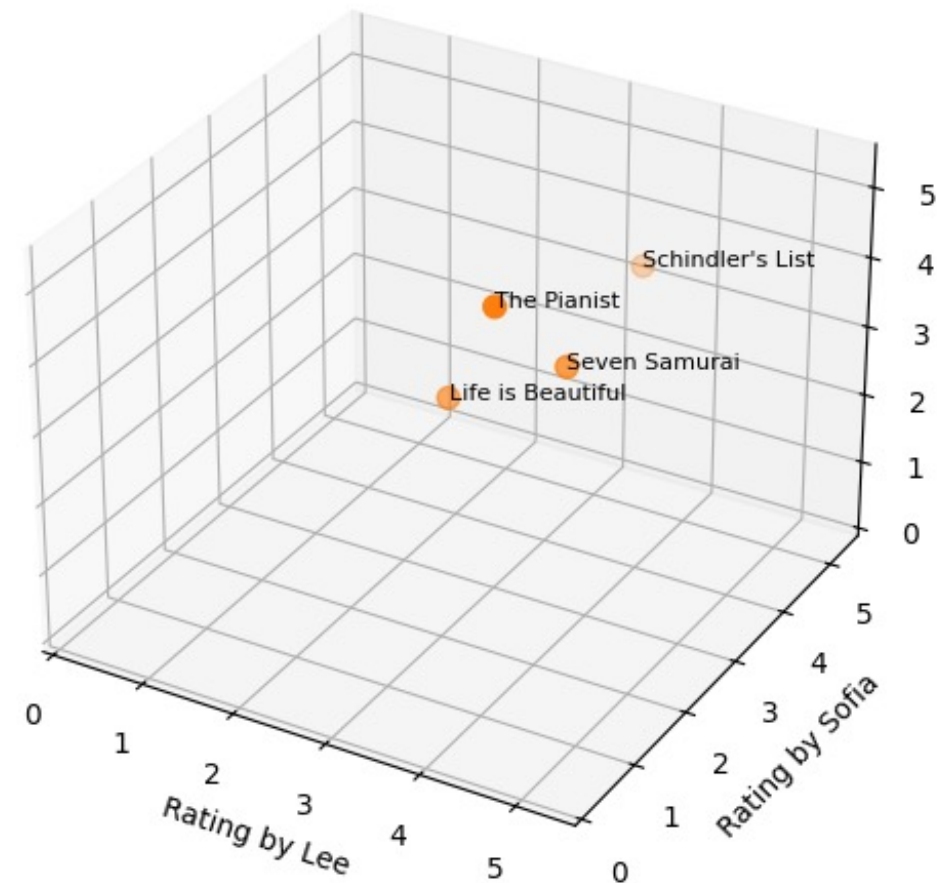
|         | Life is Beautiful | Seven Samurai | Joker | Schindler's List | The Pianist | City Of God |
|---------|-------------------|---------------|-------|------------------|-------------|-------------|
| Lee     | 2.5               | 3.5           | 3.0   | 3.5              | 3.0         | 2.5         |
| Sofia   | 3.0               | 3.5           | 1.5   | 5.0              | 3.0         | 3.5         |
| Miley   | 2.5               | 3.0           | NaN   | 3.5              | 4.0         | NaN         |
| Justina | NaN               | 3.5           | 3.0   | 4.0              | 4.5         | 3.0         |
| Donald  | 3.0               | 4.0           | 2.0   | 3.0              | 3.0         | 2.0         |
| Mickey  | 3.0               | 4.0           | NaN   | 5.0              | 3.0         | 3.5         |
| Tristan | NaN               | 4.5           | NaN   | 4.0              | NaN         | 1.0         |

**Collaborative (item-based) systems use similarities between items.** We look for films whose rating patterns are similar to ones Miley already likes. If “**Joker**” or “**City Of God**” look similar to films she rated highly, the system predicts she’ll like them too and recommends them.

# Collaborative Filtering – Item-based Approach

Films in 3D rating space (item view)

|                | Seven Samurai | Schindler's List | The Pianist |
|----------------|---------------|------------------|-------------|
| <b>Lee</b>     | 3.5           | 3.5              | 3.0         |
| <b>Sofia</b>   | 3.5           | 5.0              | 3.0         |
| <b>Miley</b>   | 3.0           | 3.5              | 4.0         |
| <b>Justina</b> | 3.5           | 4.0              | 4.5         |
| <b>Donald</b>  | 4.0           | 3.0              | 3.0         |
| <b>Mickey</b>  | 4.0           | 5.0              | 3.0         |
| <b>Tristan</b> | 4.5           | 4.0              | NaN         |



(Note: user-based CF: films as coordinates, and item-based CF: users as coordinates)

# Collaborative Filtering – Item-based Approach

To compute recommendations using this approach:

Estimate the rating for unrated item  $\hat{i}$  that has a top-N similarity to a rated item  $i$ :

$$r_{u,\hat{i}} = \frac{\sum_{i \in I} \text{sim}(\hat{i}, i) r_{u,i}}{\sum_{i \in I} \text{sim}(\hat{i}, i)}$$

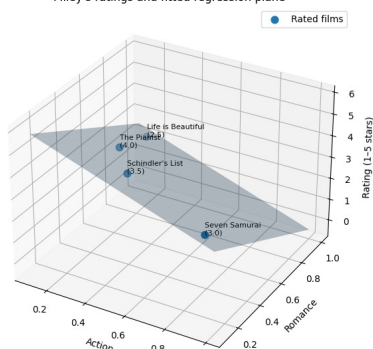
Where  $I$  is the subset of all  $N$  items similar to  $\hat{i}$

[Demo: Item-based recommendation](#)

# Recommendation Systems – Summary

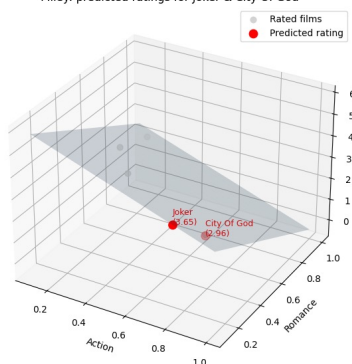
## Content-based

Miley's ratings and fitted regression plane

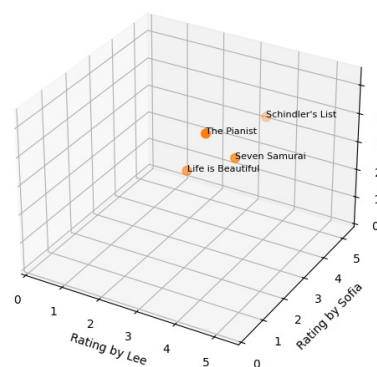


Users in 3D movie-rating space

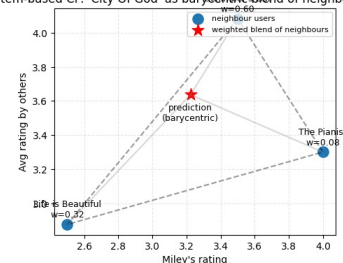
Miley: predicted ratings for Joker & City Of God



Films in 3D rating space (item view)

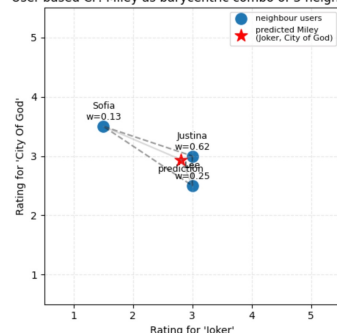


Item-based CF: 'City Of God' as barycentric blend of neighbour films



(Item-based)

User-based CF: Miley as barycentric combo of 3 neighb



User-based

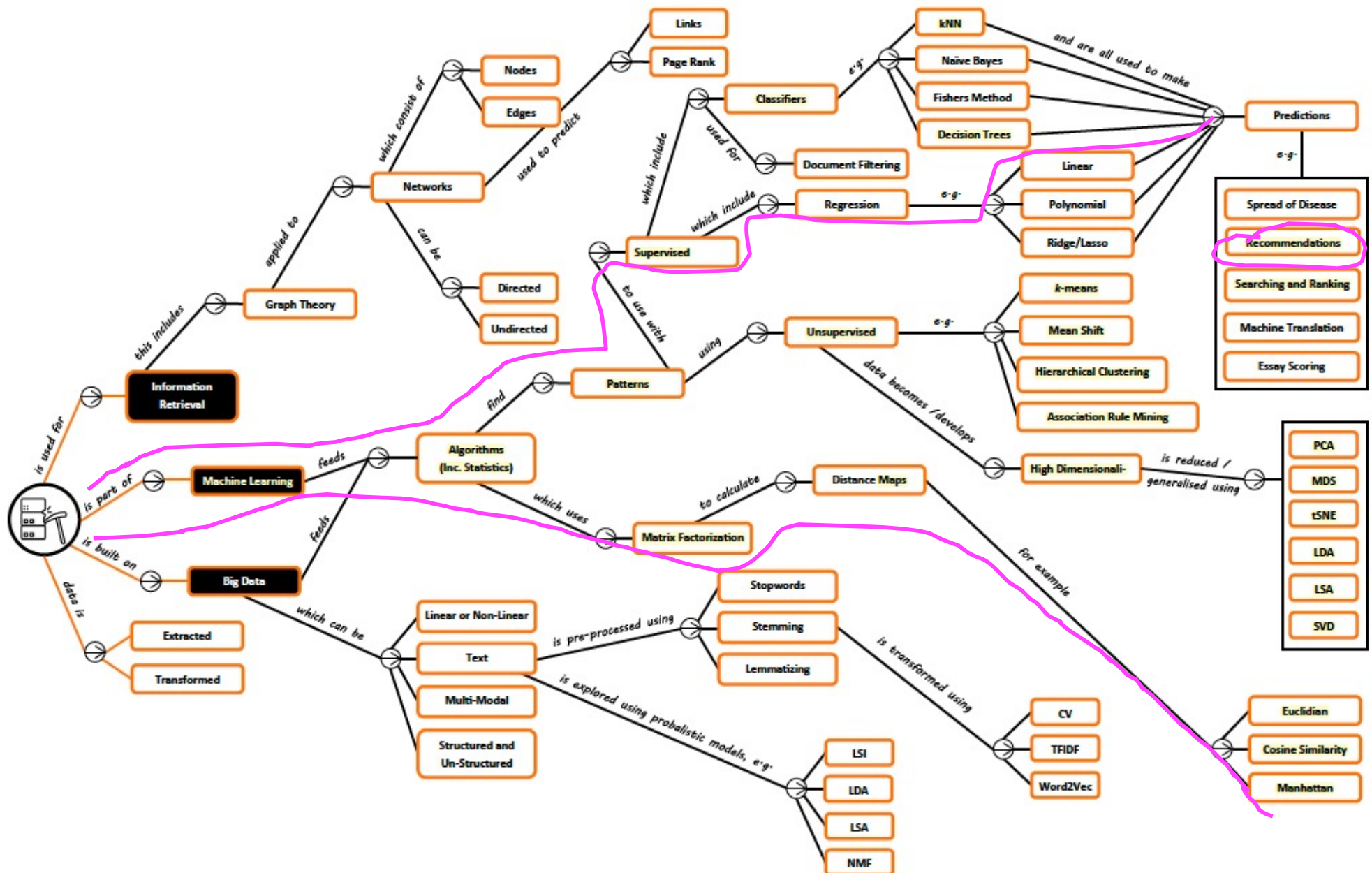
## Approach

## Big win

## Main pain point

|               |                                     |                                        |
|---------------|-------------------------------------|----------------------------------------|
| Content-based | Works for new items, few users      | Needs good features, "more of same"    |
| User-based CF | Very intuitive, serendipitous finds | Needs overlap, new users are hard      |
| Item-based CF | Fast, widely used in real systems   | New items are hard, needs many ratings |

# Recommendation Systems – Roadmap





# Recommendation Systems – Textbook

## Chapter 9

# Recommendation Systems

There is an extensive class of Web applications that involve predicting user responses to options. Such a facility is called a *recommendation system*. We shall begin this chapter with a survey of the most important examples of these systems. However, to bring the problem into focus, two good examples of recommendation systems are:

1. Offering news articles to on-line newspaper readers, based on a prediction of reader interests.
2. Offering customers of an on-line retailer suggestions about what they might like to buy, based on their past history of purchases and/or product searches.

► Mining of Massive Datasets J. Leskovec *et al*



# Recommendation Systems – Learning Outcomes

- **LO1:** Mastering fundamental concepts and mathematical calculations of content-based and user-based/item-based collaborative filtering approaches, such as (exam)
  - ❖ Measuring distances/similarities between users or items
  - ❖ Predicting the missing rating using content-based approach
  - ❖ Calculating the predicted rating with user-based/item-based collaborative filtering approach
- **LO2:** Implement basic algorithms using Python (coursework)

***Assessment hints: Multi-choice Questions (single answer: concepts, calculation etc)***

- *Textbook Exercises: textbooks (Programming + Mining)*
- *Other Exercises: <https://www-users.cse.umn.edu/~kumar001/dmbook/sol.pdf>*
- *ChatGPT or other AI-based techs*